

РЕЗЮМЕ

Тема курсовой работы

«Частное ателье»

Работа содержит: 40 страниц, 9 рисунков, 10 таблиц, 3 приложения, 10 использованных источника литературы.

Ключевые слова: ателье, частное ателье, приложение, веб-приложение, модели базы данных, базы данных.

Цель курсовой работы – углубление теоретических знаний по дисциплине «Базы данных», развитие навыков самостоятельной исследовательской работы.

Объектом исследования выступают частные ателье Республики Беларусь.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	4
1 АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ	5
1.1 Актуальность разрабатываемой базы данных	5
1.2 Описание предметной области	5
2 ПРОЕКТИРОВАНИЕ ИНФОРМАЦИОННОЙ СИСТЕМЫ.....	8
2.1 Проектирование базы данных	8
2.1.1 Разработка концептуальной модели базы данных	8
2.1.2 Разработка логической модель базы данных	10
2.1.3 Разработка физической модели базы данных	12
2.2 Моделирование функции системы (UML-нотации).....	13
2.2.1 Диаграмма вариантов использования	13
2.2.2 Диаграмма последовательности	14
2.3 Проектирование приложения	14
3 РЕАЛИЗАЦИЯ ИНФОРМАЦИОННОЙ СИСТЕМЫ	15
3.1 Характеристика средств реализации	15
3.2 Физическая реализация базы данных.....	15
3.3 Создание приложения для работы с базой данных	17
3.3.1 Слой доступа к данным.....	17
3.3.2 Слой бизнес-логики.....	17
3.3.3 Слой представления	18
3.3.4 Безопасность приложения.....	19
ЗАКЛЮЧЕНИЕ	22
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	23
ПРИЛОЖЕНИЯ.....	24

ВВЕДЕНИЕ

В современной сфере бизнеса управление данными стало одной из ключевых задач для успешной работы бизнеса. Особенно это актуально для небольших предприятий, таких как частное ателье. Данное предприятие нуждается в инструментах для обработки информации о клиентах, заказах и услугах. В связи с данными высказываниями, разработка базы данных и приложения для автоматизации и оптимизации процессов для частного ателье является актуальной задачей.

Целью данного курсового проекта является поэтапная разработка базы данных и приложения для частного ателье, которое будет включать в себя учёт клиентов, заказов и материалов. Данный проект ориентирован на создание удобного интерфейса для пользователей и для комфортной работы сотрудников. Также проект направлен на обеспечение надёжного хранения данных.

В данной работе будут изучены основные этапы проектирования баз данных такие, как разработка концептуальной, логической и физической моделей баз данных. Ожидается, что данный проект не только упростит некоторые процессы, но и повысит уровень обслуживания клиентов.

1 АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ

1.1 Актуальность разрабатываемой базы данных

В условиях современных технологий и бизнес-процессов использование баз данных становится одним из ключевых факторов успеха. Для частного ателье база данных позволяет улучшить обслуживание клиентов, сократить время на обработку заказов, автоматизировать процессы, а также в целом повысить общую продуктивность работы ателье.

1.2 Описание предметной области

Данная база данных предназначена для оптимизации и автоматизации работы частного ателье по ремонту и пошиву одежды. Она работает с такими сущностями и процессами, которые связаны с клиентами, сотрудниками, материалами, услугами и измерениями.

Клиенты являются основными потребителями услуг данного ателье. Для каждого клиента, который воспользовался частным ателье хранится следующая информация:

- Уникальный номер клиента;
- ФИО клиента;
- Контактные данные: Email и номер телефона;
- Пароль для входа в личный кабинет.

Сотрудниками являются люди, которые работают в ателье. Для каждого сотрудника должна фиксироваться следующая информация:

- Уникальный номер сотрудника;
- ФИО сотрудника;
- Контактные данные: Email и номер телефона;
- Пароль для входа в систему.

Заказы является основной сущностью, которая связывает клиентов, сотрудников и предоставляемых услуг. Каждый заказ содержит следующую

информацию:

- Уникальный номер заказа;
- Дата оформления заказа;
- Общую стоимость заказа.

Категории – это сущность, содержащая типы услуг, которые предлагает частное ателье. В категории входит такая информация, как:

- Уникальный номер категории;
- Название категории.

Услуги – это сущность, которая раскрывает более подробно каждую категорию. Услуги содержат:

- Уникальный номер услуги;
- Название услуги;
- Разновидность услуги(с какими видами одежды могут работать);
- Цена услуги.

Сущность Измерения хранит уникальный номер, а также служит для того, чтобы хранить замеры клиентов, такие как: ширина плеч, полуобхват груди, талии и бёдер; длина рукава, изделия и бокового шва.

Сущность Примерки служит для того, чтобы сотрудники ателье могли назначать дату примерки и оставлять результат примерки для заказа клиента.

Материалы – это сущность, предназначенная для хранения информации об ткани и фурнитуре, которая используется в заказе клиента. Также в этой сущности хранится следующая информация:

- Количество ткани и фурнитуры;
- Цена за использованную ткань и фурнитуру.

Ткани и Фурнитура – это две похожие друг на друга сущности, содержащие информацию о материалах, хранящихся на складе. Данные сущности содержат следующую информацию:

- Обе сущности имеют уникальный номер;
- Обе сущности имеют название материала;
- Обе сущности имеют цену за единицу (цена за метр у сущности Ткани,

цена за единицу у сущности Фурнитура);

- Обе сущности содержат информацию о количестве материала на складе.

База данных построена с помощью связей между сущностями, что позволяет организовать целостную модель данных. Основными связями являются:

- Клиенты и сотрудники связаны с заказами;
- Заказы связаны с категориями, услугами, измерениями, примерками и материалами;
- Материалы включают в себя ткань и фурнитуру.

Все эти связи обеспечены через внешние ключи.

2 ПРОЕКТИРОВАНИЕ ИНФОРМАЦИОННОЙ СИСТЕМЫ

2.1 Проектирование базы данных

Проектирование базы данных состоит из нескольких шагов, которые помогают создавать, внедрять и поддерживать системы управления данными бизнеса.

2.1.1 Разработка концептуальной модели базы данных

Концептуальная модель базы данных – это модель, визуально отображающая данные и связи предметной области в виде нарисованной диаграммы. Создаётся концептуальная модель для того, чтобы в дальнейшем проектировании базы данных её можно было перевести в реляционную модель.

Чтобы приступить к построению концептуальной модели нужно идентифицировать исходные сущности:

- Клиенты;
- Сотрудники;
- Заказы;
- Категории;
- Услуги;
- Измерения;
- Примерки;
- Материалы для заказа;
- Материалы на складе.

Далее нужно определить взаимосвязи между объектами. Можем определить следующие взаимосвязи:

- Клиент может сделать несколько заказов. Каждый заказ принадлежит определённому клиенту;
- Сотрудник может обслуживать несколько заказов. И каждый заказ может обслуживаться только тем сотрудником, который работает над этим заказом;

- Заказ может относиться только к одной категории. Одна категория может относиться к разным заказам;
- В заказ может входить только 1 услуга. Одна услуга может входить в разные заказы;
- Примерка назначается для каждого заказа. Для одного заказа может назначаться несколько примерок;
- Измерения имеются у каждого заказа. Один заказ может иметь несколько измерений;
- Материалы для заказа содержатся в одном заказе. В одном заказе может содержаться несколько материалов для заказа;
- Материалы на складе могут использоваться в нескольких материалах для заказов. В материалах для заказа может содержаться несколько материалов со склада.

Мы можем определить следующие атрибуты для объектов:

Клиент: ID клиента (первичный ключ), ФИО (фамилия, имя, отчество), контактные данные (контактный телефон, email), пароль.

Сотрудник: ID сотрудника (первичный ключ), ФИО (фамилия, имя, отчество), контактные данные (контактный телефон, email), пароль.

Заказы: ID заказа (первичный ключ), дата оформления, общая стоимость.

Категории: ID категории (первичный ключ), название.

Услуги: ID услуги (первичный ключ), название, разновидность, цена.

Измерения: ID замера (первичный ключ), ширина (плеч), полуобхват (груди, талии, бедер), длина (рукава, изделия, бокового шва).

Примерки: ID примерки (первичный ключ), дата примерки, результат.

Материалы для заказа: ID материала для заказа (первичный ключ), количество, цена.

Материалы на складе: ID материала на складе (первичный ключ), название, цена, количество. Основываясь на сущностях и связях, построим диаграмму, показанную на рисунке 2.1.1.

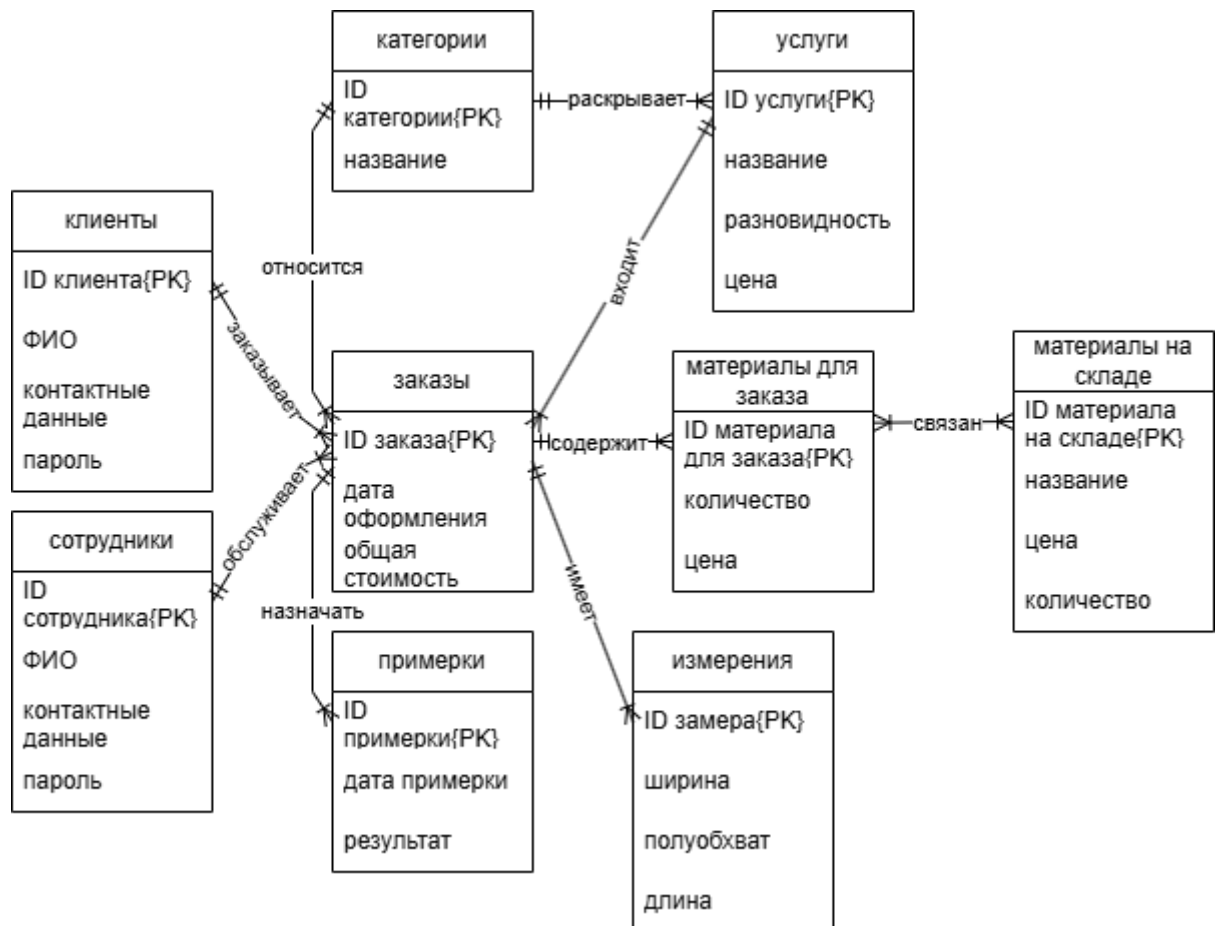


Рисунок 2.1.1 – Концептуальная модель базы данных

2.1.2 Разработка логической модель базы данных

Логическая модель базы данных – это расширенная концептуальная модель базы данных. Она включает в себя все сущности, атрибуты, ключи и взаимосвязи, которые представляют бизнес-информацию и определяют бизнес правила.

Для каждой сущности распишем и добавим атрибуты:

Клиенты: разобьём ФИО на фамилию, имя, отчество; разобьём контактные данные на email и контактный телефон.

Сотрудники: разобьём ФИО на фамилию, имя, отчество; разобьём контактные данные на email и контактный телефон.

Заказы: добавим внешние ключи: ID клиента, ID сотрудника, ID категории, ID услуги.

Услуги: добавим внешний ключ ID категории.

Примерки: добавим внешний ключ ID заказа.

Замеры: переименуем ширину в ширину плеч, разобьём полуобхват на полуобхват груди, полуобхват талии, полуобхват бедер; разобьём длину на длину рукава, длину изделия, длину бокового шва; добавим внешний ключ ID заказа.

Материалы для заказа: разобьём количество материала на количество ткани и количество фурнитуры, добавим внешние ключи: ID ткани, ID фурнитуры, ID заказа.

Материалы на складе разобьём на 2 сущности: ткань и фурнитура.

Основываясь на новых данных, построим логическую модель базы данных, показанную на рисунке 2.1.2.

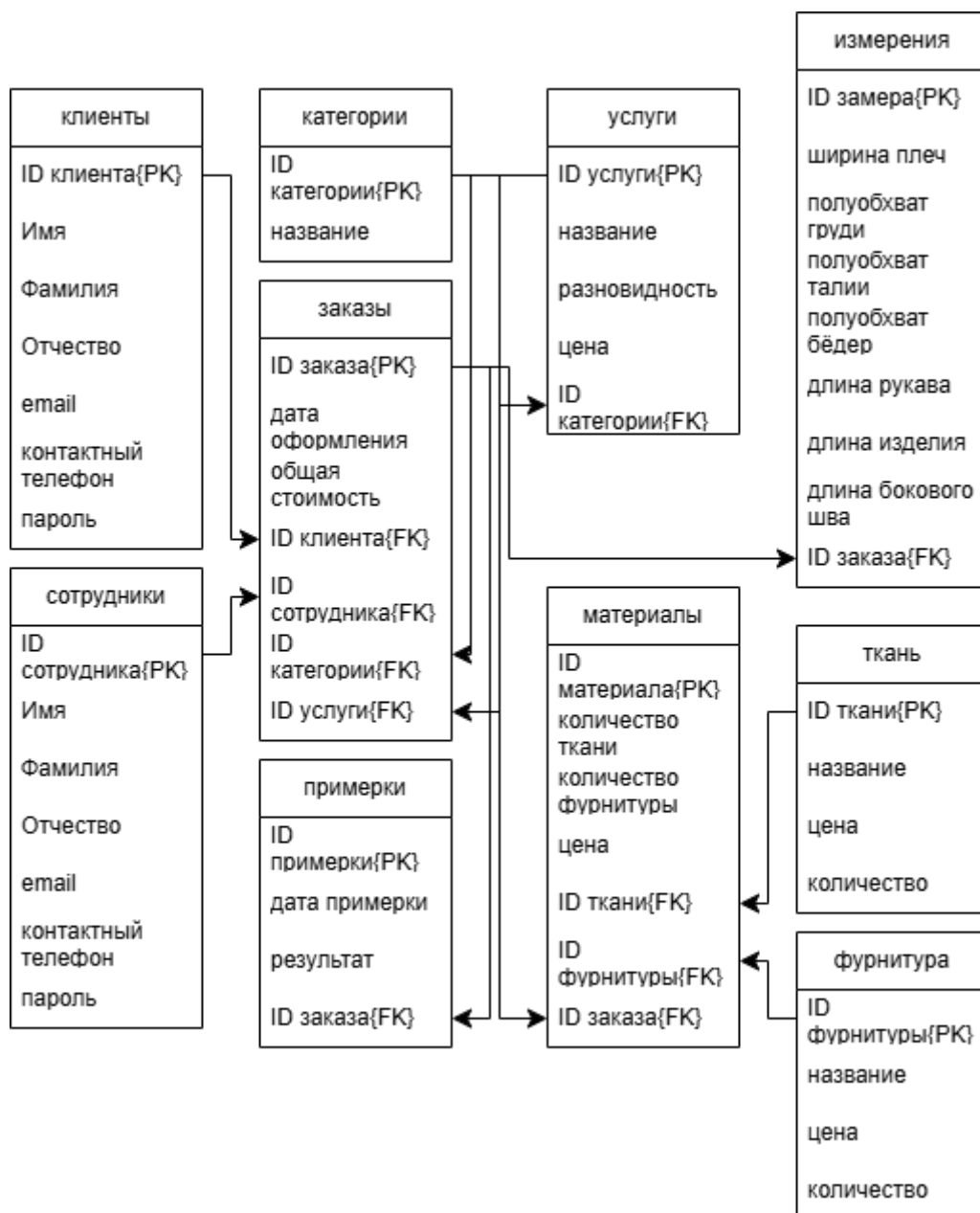


Рисунок 2.1.2 – Логическая модель базы данных

2.1.3 Разработка физической модели базы данных

Физическая модель базы данных – это описание того, как данные хранятся в конкретной системе управления базами данных (СУБД). В физической модели указываются типы данных для всех сохранённых объектов. Также для нее могут изменяться имена сущностей и атрибутов, чтобы они соответствовали поддерживаемым платформой форматами. Для дальнейшей работы мы выбираем СУБД MySQL. Из изменений следует отметить, что, для таблицы

Категории были добавлены два новых атрибута: изображение и ссылка. Нужны они для приложения. Подробную физическую модель можно рассмотреть в приложении 1.

2.2 Моделирование функции системы (UML-нотации)

UML (Unified Modeling Language) – стандартный язык для описания, визуализации, проектирования и документации элементов информационных систем.

2.2.1 Диаграмма вариантов использования

Диаграмма вариантов использования представляет роли пользователей и действия, которые они выполнять в системе. Основные роли и действия можно посмотреть на рисунке 2.2.1.



рисунок 2.2.1 – Диаграмма вариантов использования

2.2.2 Диаграмма последовательности

Диаграмма последовательности используется для визуализации взаимодействий между объектами в системе.

Основные компоненты диаграммы последовательности включают обозначения объектов, вертикальные «линии жизни», которые представляют течение времени, прямоугольники, отражающие действия объектов или выполнение ими определённых функций, и стрелки, указывающие на обмен сигналами или сообщениями между объектами. Диаграмму последовательности для каждого действия рассмотрено в приложении 2.

2.3 Проектирование приложения

Для проектирования приложения, которое обеспечивает взаимодействие с базой данных нужно определить следующие компоненты:

Интерфейс пользователя;

- Интерфейс клиентов для регистрации, авторизации и оформления заказа;
- Интерфейс сотрудников для удобной работы с заказами клиентов.

Логика приложения;

- Взаимодействие с базой данных через SQL запросы;
- Использование триггеров для автоматизации процессов.

Архитектура приложения.

3 РЕАЛИЗАЦИЯ ИНФОРМАЦИОННОЙ СИСТЕМЫ

3.1 Характеристика средств реализации

Для базы данных частного ателье будет использовано СУБД MySQL.

Для удобной работой работы с базой данных мы будем использовать phpmyadmin.

Для обработки запросов и взаимодействия с базой данных будет использоваться PHP.

Для интерфейса нашего приложения будут использоваться html, css и javascript.

3.2 Физическая реализация базы данных

Физическая модель базы данных – это модель данных, которая определяет, каким образом представляются данные, и содержит все детали, необходимые СУБД для создания базы данных.

В phpmyadmin с помощью встроенных инструментов и SQL запросов создаём базу данных. Запрос с создание рассмотрим в примере 3.2.1.

Пример 3.2.1 – Создание базы данных

```
CREATE DATABASE private_atelie;
```

Далее, после создания базы данных, создадим сами таблицы, индексы и триггеры для нашей базы данных. Создание данных объектов рассмотрим в примерах 3.2.2 – 3.2.4.

Пример 3.2.2 – Создание таблиц

```
CREATE TABLE `material` (  
  `materialID` int NOT NULL AUTO INCREMENT,
```

```
`tex_q` decimal(10,2) DEFAULT 0.00,
`acc_q` int DEFAULT 0,
`material_price` decimal(10,2) DEFAULTL 0.00,
`textileID` int DEFAULT NULL,
`accessoriesID` int DEFAULT NULL,
`orderID` int DEFAULT NULL
);
```

Пример 3.2.3 – Создание индексов

```
ALTER TABLE `material` ADD PRIMARY KEY(`materialID`),
ADD FOREIGN KEY(`orderID`) REFERENCES corder(`orderID`),
ADD FOREIGN KEY(`accessoriesID`) REFERENCES
accessories(`accessoriesID`),
ADD FOREIGN KEY(`textileID`) REFERENCES textile(`textileID`);
```

Пример 3.2.4 – Создание триггеров

```
DELIMITER $$
CREATE TRIGGER `after_material_update_tex_q` AFTER UPDATE ON
`material` FOR EACH ROW BEGIN
    IF NEW.tex_q != OLD.tex_q AND NEW.tex_q > 0 THEN
        UPDATE textile
        SET tex_quantity = tex_quantity - (NEW.tex_q - OLD.tex_q)
        WHERE textileID = NEW.textileID;
    END IF;
END
$$
DELIMITER;
```

Подробная физическая реализация базы данных представлена в приложении 3.

3.3 Создание приложения для работы с базой данных

3.3.1 Слой доступа к данным

Доступ к данным для клиентов и сотрудников предоставляется через веб-приложение. Если пользователь не авторизован в системе, то он может только просматривать атрибут `categories_name` из таблицы `categories`, а также пользователь может просматривать атрибуты `service_name`, `service_comm` и `service_price` из таблицы `service`.

Если пользователь авторизован и является клиентом, то он также, как и не авторизованный пользователь может просматривать некоторые атрибуты из таблиц `categories` и `service`, кроме этого, у него появляется доступ к личному кабинету с атрибутами `first_name`, `last_name`, `third_name`, `email` и `phone_num` этого пользователя. Также клиенту даётся доступ на добавление новой записи в таблицу `corder`, то есть он может сделать заказ.

Если пользователь авторизован и является сотрудником к вышеперечисленным доступам добавляется управление заказами, то есть доступ к добавлению, изменению данных из таблицы `corder`. Также сотрудник имеет доступ к таблицам `measurements` и `material`. Для таблиц `textile`, `accessories` и `fitting` сотрудник может только добавлять и удалять записи.

3.3.2 Слой бизнес-логики

Бизнес-логика – это набор правил, которым подчиняются объекты, сущности, классы и данные внутри программы.

Для подсчёта цены и количества материала, который будет использован в заказе клиента, созданы триггеры, срабатывающие тогда, когда сотрудник изменяет поля `tex_q` и `acc_q` в таблице `material`.

Также есть триггер, высчитывающий цену за заказ. Срабатывает он тогда, когда полю `fitting_results` из таблицы `fitting` присваивается одно из указанных в триггере значений.

После авторизации пользователя его переводит в личный кабинет. Если пользователь не авторизован, то в личный кабинет он попасть не сможет.

При добавлении, изменении и удалении сотрудником некоторой информации, эти изменения видны в приложении.

Если пользователь не заполнил все нужные поля, приложение предупреждает об этом.

3.3.3 Слой представления

Слой представления – это слой, который взаимодействует непосредственно с пользователем. Он отвечает за представление информации и адаптируется под различные платформы, например, веб и мобильные приложения, без изменения бизнес-логики.

Для интерфейса приложения используются такие технологии, как `html`, `css` и `javascript`.

Предположительный возраст клиентов не определен, поэтому приложение должно быть привлекательным и удобным для людей разных возрастов.

Цвета должны быть приятные глазу, не слишком яркие, так как большинство пользователей отталкивает «буйство красок». Также цветовая палитра должна состоять из оттенков, которые сочетаются между собой. Для нашего приложения будет использоваться цветовая палитра, состоящая из следующих оттенков: `#FFFFFF`, `#F8F9FA`, `#212529`. Данная палитра содержит малое количество цветов, но они хорошо сочетаются друг с другом и не «мозолят» глаз.

Мы будем использовать `Bootstrap`. `Bootstrap` – это фреймворк для создания сайтов и веб-приложений с открытым исходным кодом. Включает в

себя html и css шаблоны оформления для типографии, веб-форм, кнопок, меток, блоков навигации и прочих компонентов веб-интерфейса, включая javascript-расширения. Помимо готовых шаблонов из Bootstrap мы будем использовать свои шаблоны.

Интерфейс приложения не должен быть перегружен, но также быть одновременно информативным и понятным для пользователя.

3.3.4 Безопасность приложения

Безопасность приложения – один из самых важных аспектов, который обязательно должен быть учтён при разработке приложения. Рассмотрим основные системы безопасности, использованные в нашем приложении.

- Фильтрация ввода данных помогает улучшить стабильность, безопасность и внутреннюю структуру кода. Проверка ввода является одним из важных пунктов обеспечения безопасности кода. Использование фильтрации ввода помогает предотвращению XSS атак. В примере 3.3.4.1 рассмотрена одна из нескольких существующих фильтров

Пример 3.3.4.1 – Фильтрация ввода

```
$accessories = trim($_POST['accname']),  
FILTER_SANITIZE_SPECIAL_CHARS);
```

- При написании SQL запросов следует использовать подготовленные выражения. Такие запросы помогут избежать CSRF атаки и SQL инъекций.

Использование подготовленного выражения можно посмотреть в примере 3.3.4.2.

Пример 3.3.4.2 – Использование подготовленных запросов

```
$mysql = "INSERT INTO accessories(acc_name, acc_unit_price,
```

```
acc_quantity) VALUES(?, ?, ?)";
$query = $pdo->prepare($mysql);
$query->execute([$accessoriesname, $accessoriesprice,
$accessoriesquantity]);
```

– Одной из угроз для веб-приложения является внедрение зловредного кода. Для защиты приложения от таких угроз, нужно использовать специализированные функции, которые предотвращают теги в html entities. Одну из таких функций можно посмотреть в примере 3.3.4.3.

Пример 3.3.4.3 – Защита от внедрения кода

```
<th scope="col">' . htmlspecialchars($row['service_name']) . '</th>
```

– Хэширование паролей – приоритетный фактор безопасности, который учитывают в разработке приложения или службы, которые принимают пароли пользователей. Без хеширования пароли, которые хранятся, украдут, то немедленно применят данные пароли для входа в учётные записи, а также могут войти в учётные записи в других службах, если пользователи не установили уникальные пароли. Для хэширования паролей в PHP следует использовать функцию password_hash(). Для проверки пароля используется функция password_verify(). Примеры хэширования и проверки паролей указаны в примерах 3.3.4.4 и 3.3.4.5.

Пример 3.3.4.4 – Хэширования пароля

```
$hashedpass = password_hash($passfirst, PASSWORD_DEFAULT);
```

Пример 3.3.4.5 – Проверка пароля

```
$mysql = "SELECT * FROM clients WHERE email = ? AND phone_num =
?";
$query = $pdo->prepare($mysql);
```

```
$query->execute([$email, $phonenummer]);  
if($user) {  
    if(password_verify($pass, $user['passwd'])) {  
        // выполняемый код  
    }  
}
```

ЗАКЛЮЧЕНИЕ

В ходе работы по подготовке курсового проекта были углублены знания по проектированию базы данных и изучены технологии по созданию веб-приложений. Из проделанной работы были углублены и изучены знания об создании базы данных, основные принципы работы с ними, формирование SQL запросов, создании интерфейса html и css, языки программирования php и javascript а также была изучена программа для администрирования базы данных phpmyadmin.

Данный проект не является идеальным, но процесс его создания помог оптимально применить мои теоретические и практические знания.

В будущем проект будет дорабатываться и может быть выйдет на большой рынок.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

- 1 Habr. – Электронный ресурс. – Режим доступа:
<https://habr.com/ru/articles/556790/>.
- 2 Habr. – Электронный ресурс. – Режим доступа:
<https://habr.com/ru/articles/566218/>.
- 3 Habr. – Электронный ресурс. – Режим доступа:
<https://habr.com/ru/articles/814769/>.
- 4 Блог программиста. – Электронный ресурс. – Режим доступа: <https://pro-prof.com/archives/2594/>.
- 5 Яндекс практикум. – Электронный ресурс. – Режим доступа:
<https://practicum.yandex.ru/blog/sequence-diagram/>.
- 6 Metanit. – Электронный ресурс. – Режим доступа:
<https://metanit.com/sql/mysql/>.
- 7 PHP. – Электронный ресурс. – Режим доступа:
<https://www.php.net/manual/en/index.php>.
- 8 MySQL. – Электронный ресурс. – Режим доступа:
<https://dev.mysql.com/doc/>.
- 9 Новиков, Б.А. Основы технологий баз данных / Б.А. Новиков, Е.А. Горшкова. – Москва: ДМК, 2019.
- 10 Попова-Коварцева, Д.А. Основы проектирования баз данных / Д.А. Попова-Коварцева, Е.В Сопченко. – Самара: Издательство Самарского университета, 2019.

ПРИЛОЖЕНИЯ

ПРИЛОЖЕНИЕ 1

Таблица 1 – Клиент

Имя	Тип	NUL L	По умолчан ию	Индек с	Дополнительн о	Коммента рии
clientID	Int	Нет	Нет		AUTO_INCRE MENT	первичный ключ
First_na me	Varchar(5 0)	Нет	Нет			имя
Last_na me	Varchar(5 0)	Нет	Нет			фамилия
Third_na me	Varchar(5 0)	Да	NULL			отчество
Email	Varchar(5 0)	Нет	Нет	UNIQ UE		email
Phone_n um	Varchar(5 0)	Нет	Нет	UNIQ UE		Контактны й телефон
passwd	Varchar(2 55)	Нет	Нет			пароль

Таблица 2 – Сотрудники

Имя	Тип	NUL L	По умолчан ию	Индек с	Дополнительн о	Коммента рии
employe eID	Int	Нет	Нет		AUTO_INCRE MENT	первичный ключ
First_na me	Varchar(5 0)	Нет	Нет			имя
Last_na me	Varchar(5 0)	Нет	Нет			фамилия

Third_name	Varchar(50)	Нет	Нет			отчество
Email	Varchar(50)	Нет	Нет	UNIQUE		email
Phone_num	Varchar(50)	Нет	Нет	UNIQUE		Контактный телефон
passwd	Varchar(255)	Нет	Нет			пароль

Таблица 3 – Заказы

Имя	Тип	NULL	По умолчанию	Индекс	Дополнительно	Комментарии
orderID	Int	нет	нет		AUTO_INCREMENT	первичный ключ
Order_date	Date	нет	нет			Дата заказа
Order_price	Decimal(10, 2)	да	0.00			Цена заказа
categoriesID	Int	да	NULL			Внешний ключ
ClientID	Int	да	NULL			Внешний ключ
employeeID	Int	да	NULL			Внешний ключ
serviceID	Int	да	NULL			Внешний ключ

Таблица 4 – Примерки

Имя	Тип	NUL L	По умолчан ию	Инде кс	Дополнительно	Комментар ии
fittingID	Int	Нет	Нет		AUTO_ENCRE MENT	первичный ключ
Fit_results	Varchar(1 00)	Да	Не начат			Результат примерки
Plane_date	Date	Да	NULL			Запланирова нная дата примерки
orderID	Int	да	NULL			Внешний ключ

Таблица 5 – Категории

Имя	Тип	NU LL	По умолчан ию	Инде кс	Дополнительн о	Коммента рии
categoriesID	Int	Нет	Нет		AUTO_ENCRE MENT	первичны й ключ
Categories_ name	Varchar(50)	Нет	Нет			Первичны й ключ
Categories_ pic	Varchar(50)	Нет	Нет			Изображе ние категории
link	Varchar(50)	нет	нет			ссылка

Таблица 6 – Услуги

Имя	Тип	NULL	По умолчанию	Индекс	Дополнительно	Комментарии
serviceID	Int	Нет	Нет		AUTO_INCREMENT	первичный ключ
Service_name	Varchar(50)	Да	NULL			название
Service_comm	Varchar(50)	Да	NULL			Разновидность
Service_price	Decimal(10, 2)	Да	NULL			Цена
categoriesID	Int	да	NULL			Внешний ключ

Таблица 7 – Измерения

Имя	Тип	NULL	По умолчанию	Индекс	Дополнительно	Комментарии
measID	Int	Нет	Нет		AUTO_INCREMENT	первичный ключ
Shirina_plech	Varchar(50)	Да	NULL			Ширина
Poluobhvat_grudi	Varchar(50)	Да	NULL			полуобхват
Dlina_rukava	Varchar(50)	Да	NULL			Длина
Dlina_izdeliya	Varchar(50)	Да	NULL			Длина
Dlina_bokovogo_shva	Varchar(50)	Да	NULL			Длина
Poluobhvat_talii	Varchar(50)	Да	NULL			Полуобхват

Poluobhvat_beder	Varchar (50)	Да	NULL			Полуобхват
orderID	Int	Да	NULL			Внешний ключ

Таблица 8 – Материалы

Имя	Тип	NULL	По умолчанию	Индекс	Дополнительно	Комментарии
materialID	Int	Нет	Нет		AUTO_INCREMENT	первичный ключ
Tex_q	Decimal(10, 2)	Да	0.00			Количество ткани
Acc_q	Int	Да	0			Количество фурнитуры
Material_price	Decimal(10, 2)	Да	0.00			Цена материала
textileID	Int	Да	0			Внешний ключ
accessories ID	Int	Да	0			Внешний ключ
orderID	Int	Да	NULL			Внешний ключ

Таблица 9 – Ткани

Имя	Тип	NULL	По умолчанию	Индекс	Дополнительно	Комментарии
-----	-----	------	--------------	--------	---------------	-------------

textileID	Int	Нет	Нет		AUTO_ENCRE MENT	первичный ключ
Tex_name	Varchar(1 00)	Нет	Нет			Название ткани
Tex_unit_ price	Decimal(10, 2)	Нет	Нет			Цена за метр
Tex_quant ity	Decimal(10, 2)	да	0			Количеств о

Таблица 10 – Фурнитура

Имя	Тип	NU LL	По умолчан ию	Инде кс	Дополнительно	Коммента рии
accessorie sID	Int	Нет	Нет		AUTO_ENCRE MENT	первичный ключ
acc_name	Varchar(1 00)	Нет	Нет			Название фурнитур ы
acc_unit_p rice	Decimal(10, 2)	Нет	Нет			Цена за единицу
acc_quant ity	int	да	0			Количеств о

ПРИЛОЖЕНИЕ 2

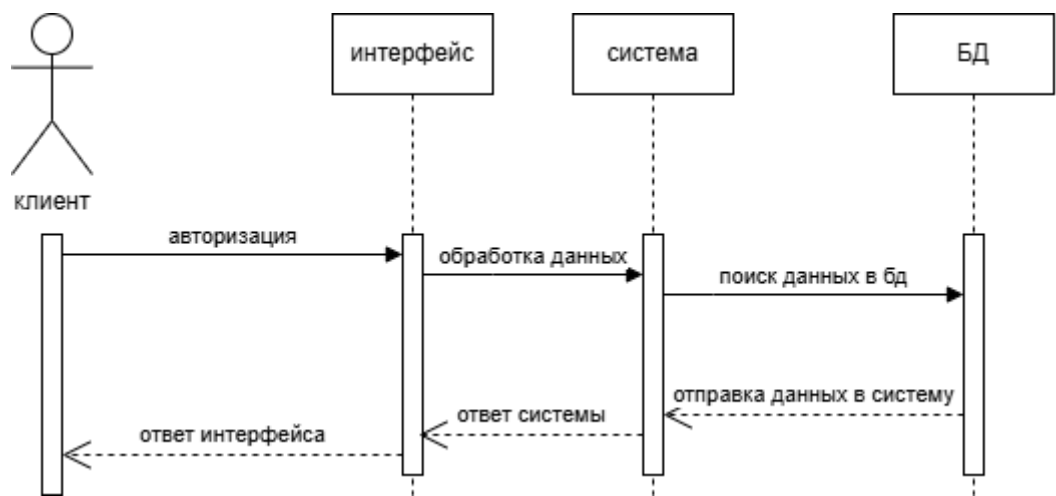


Рисунок 1 – Регистрация пользователя

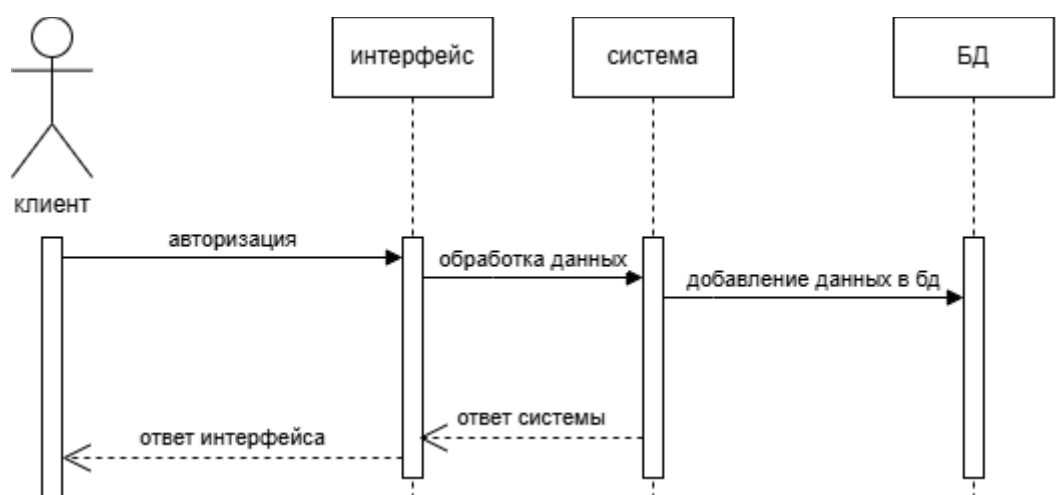


Рисунок 2 – Авторизация пользователя

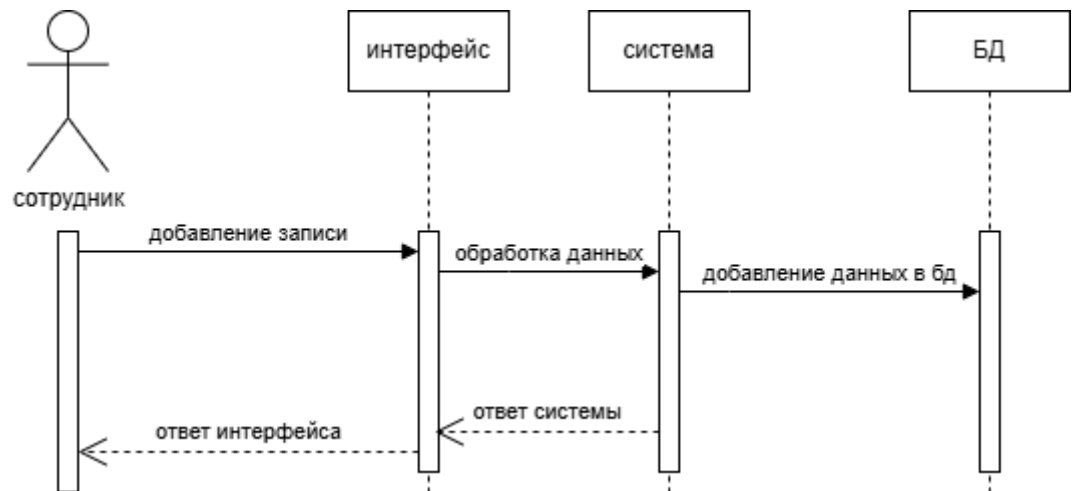


Рисунок 3 – Оформление заказа

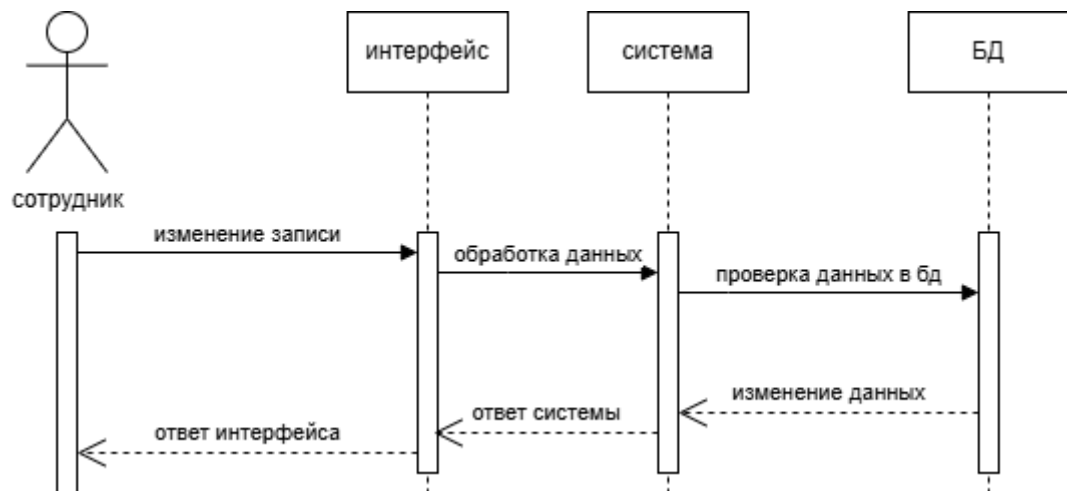


Рисунок 4 – Добавление записи

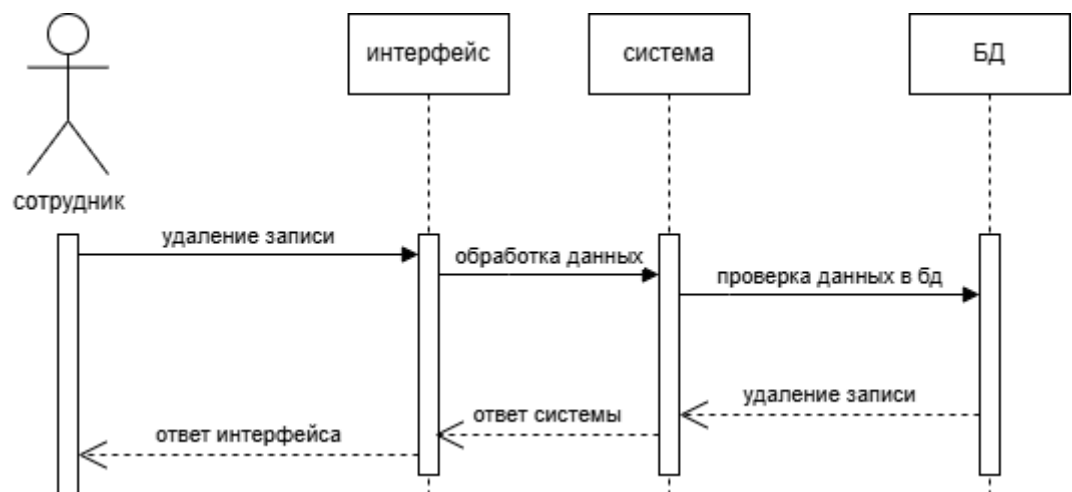


Рисунок 5 – Изменение записи

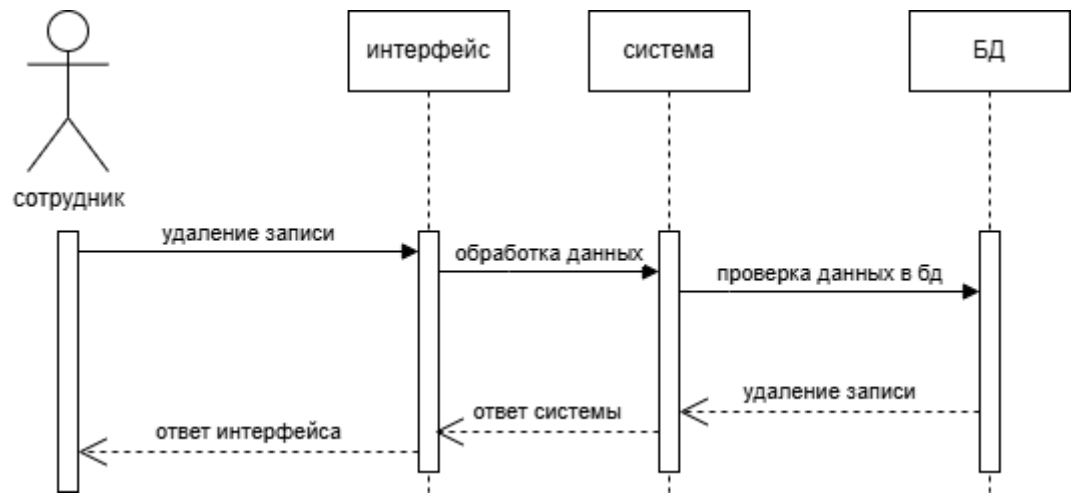


Рисунок 6 – Удаление записи

ПРИЛОЖЕНИЕ 3

Список 1 - Таблицы

```
--  
-- структура таблицы `clients`  
--  
  
CREATE TABLE `clients` (  
  `clientID` int NOT NULL AUTO INCREMENT,  
  `first_name` varchar(50) NOT NULL,  
  `last_name` varchar(50) NOT NULL,  
  `third_name` varchar(50) DEFAULT NULL,  
  `email` varchar(50) NOT NULL UNIQUE,  
  `phone_num` varchar(50) NOT NULL UNIQUE,  
  `passwd` varchar(255) NOT NULL,  
);  
  
--  
-- структура таблицы `employee`  
--  
  
CREATE TABLE `employee` (  
  `clientID` int NOT NULL AUTO INCREMENT,  
  `first_name` varchar(50) NOT NULL,  
  `last_name` varchar(50) NOT NULL,  
  `third_name` varchar(50) NOT NULL,  
  `email` varchar(50) NOT NULL UNIQUE,  
  `phone_num` varchar(50) NOT NULL UNIQUE,  
  `passwd` varchar(255) NOT NULL  
);  
  
--  
-- структура таблицы `corder`  
--  
  
CREATE TABLE `corder` (  
  `orderID` int NOT NULL AUTO INCREMENT,  
  `order_date` date NOT NULL,
```

```
`order_price` decimal(10,2) DEFAULT 0.00,
`categoriesID` int DEFAULT NULL,
`clientID` int DEFAULT NULL,
`employeeID` int DEFAULT NULL,
`serviceID` int DEFAULT NULL
);
```

```
--
-- структура таблицы `categories`
--
```

```
CREATE TABLE `categories` (
`categoriesID` int NOT NULL AUTO INCREMENT,
`categories_name` varchar(50) NOT NULL,
`categories_pic` varchar(255) NOT NULL,
`link` varchar(255) NOT NULL
);
```

```
--
-- структура таблицы `fitting`
--
```

```
CREATE TABLE `fitting` (
`fittingID` int NOT NULL AUTO_INCREMENT,
`fit_results` varchar(100) DEFAULT `изменений не требуется`,
`plane_date` date DEFAULT NULL,
`orderID` int DEFAULT NULL
);
```

```
--
-- структура таблицы `service`
--
```

```
CREATE TABLE `service` (
`serviceID` int NOT NULL AUTO INCREMENT,
`service_name` varchar(50) DEFAULT NULL,
`service_comm` varchar(150) DEFAULT NULL,
`service_price` decimal(10,2) DEFAULT 0.00,
`categoriesID` int DEFAULT NULL
);
```

--

-- структура таблицы `measurements`

--

```
CREATE TABLE `measurements` (
  `measID` int NOT NULL AUTO INCREMENT,
  `shirina_plech` varchar(50) DEFAULT `не измерено`,
  `poluobhvat_grudi` varchar(50) DEFAULT `не измерено`,
  `dlina_rukava` varchar(50) DEFAULT `не измерено`,
  `dlina_izdeliya` varchar(50) DEFAULT `не измерено`,
  `dlina_bokovogo_shva` varchar(50) DEFAULT `не измерено`,
  `poluobhvat_talii` varchar(50) DEFAULT `не измерено`,
  `poluobhvat_beder` varchar(50) DEFAULT `не измерено`,
  `orderId` int DEFAULT NULL
);
```

--

-- структура таблицы `material`

--

```
CREATE TABLE `material` (
  `materialID` int NOT NULL AUTO INCREMENT,
  `tex_q` decimal(10,2) DEFAULT 0.00,
  `acc_q` int DEFAULT 0,
  `material_price` decimal(10,2) DEFAULT 0.00,
  `textileID` int DEFAULT NULL,
  `accessoriesID` int DEFAULT NULL,
  `orderId` int DEFAULT NULL
);
```

--

-- структура таблицы `textile`

--

```
CREATE TABLE `textile` (
  `textileID` int NOT NULL AUTO INCREMENT,
  `tex_name` varchar(100) NOT NULL,
  `tex_unit_price` decimal(10,2) NOT NULL,
  `tex_quantity` decimal(10,2) NOT NULL
```

```
);
--
-- структура таблицы `accessories`
--

CREATE TABLE `accessories` (
  `accessoriesID` int NOT NULL AUTO INCREMENT,
  `acc_name` varchar(100) NOT NULL,
  `acc_unit_price` decimal(10,2) NOT NULL,
  `acc_quantity` int NOT NULL
);
```

Список 2 – Индексы

```
--
-- индексы таблицы `accessories`
--

ALTER TABLE `accessories` ADD PRIMARY KEY(`accessoriesID`);

--
-- индексы таблицы `categories`
--

ALTER TABLE `categories` ADD PRIMARY KEY(`categoriesID`);

--
-- индексы таблицы `clients`
--

ALTER TABLE `clients` ADD PRIMARY KEY(`clientID`);

--
-- индексы таблицы `employee`
--

ALTER TABLE `employee` ADD PRIMARY KEY(`employeeID`);

--
-- индексы таблицы `fitting`
```

```

--

ALTER TABLE `fitting` ADD PRIMARY KEY(`fittingID`),
ADD FOREIGN KEY(`orderID`) REFERENCES corder(orderID);
--
-- индексы таблицы `material`
--

ALTER TABLE `material` ADD PRIMARY KEY(`materialID`),
ADD FOREIGN KEY(`orderID`) REFERENCES corder(`orderID`),
ADD FOREIGN KEY(`accessoriesID`) REFERENCES
accessories(`accessoriesID`),
ADD FOREIGN KEY(`textileID`) REFERENCES textile(`textileID`);

--
-- индексы таблицы `measurements`
--

ALTER TABLE `measurements` ADD PRIMARY KEY(`measID`),
ADD FOREIGN KEY(`orderID`) REFERENCES corder(`orderID`);

--
-- индекс таблицы `service`
--

ALTER TABLE `service` ADD PRIMARY KEY(`serviceID`),
ADD FOREIGN KEY(`categoriesID`) REFERENCES
categories(`categoriesID`);

--
-- индекс таблицы `textile`
--

ALTER TABLE `textile` ADD PRIMARY KEY(`textileID`);

```

Список 3 – Триггеры

```

--
-- триггеры `corder`
--

```

```

DELIMITER $$
CREATE TRIGGER `before_insert_corder` BEFORE INSERT ON `corder`
FOR EACH ROW BEGIN
    DECLARE random randomEmployeeID INT;
    SELECT employeeID
    INTO randomEmployeeID
    FROM employee
    ORDER BY RAND()
    LIMIT 1;
    SET NEW.employeeID = randomEmployeeID;
END
$$
DELIMITER;

--
-- триггеры `fitting`
--

DELIMITER $$
CREATE TRIGGER `after_fitting_update` AFTER UPDATE ON `fitting`
FOR EACH ROW BEGIN
    DECLARE material_total DECIMAL(10,2) DEFAULT 0.00;
    DECLARE service_total DECIMAL(10,2) DEFAULT 0.00;
    IF NEW.fit_results IN ('выполнен', 'завершено', 'готов') THEN
        SELECT SUM(material_price) INTO material_total
        FROM material
        WHERE orderID = NEW.orderID;
        SELECT service_price INTO service_total
        FROM service
        WHERE serviceID = (
            SELECT serviceID FROM corder WHERE orderID =
NEW.orderID
        );
        UPDATE corder
        SET order_price = IFNULL(material_total, 0) +
IFNULL(service_total, 0)
        WHERE orderID = NEW.orderID;
    END IF;
END

```

```

$$
DELIMITER;

--
-- триггеры `material`
--

DELIMITER $$
CREATE TRIGGER `after_material_update_acc_q` AFTER UPDATE ON
`material` FOR EACH ROW BEGIN
    IF NEW.acc_q != OLD.acc_q AND NEW.acc_q > 0 THEN
        UPDATE accessories
        SET acc_quantity = acc_quantity - (NEW.acc_q - OLD.acc_q)
        WHERE accessoriesID = NEW.accessoriesID;
    END IF;
END
$$
DELIMITER;
DELIMITER $$
CREATE TRIGGER `after_material_update_tex_q` AFTER UPDATE ON
`material` FOR EACH ROW BEGIN
    IF NEW.tex_q != OLD.tex_q AND NEW.tex_q > 0 THEN
        UPDATE textile
        SET tex_quantity = tex_quantity - (NEW.tex_q - OLD.tex_q)
        WHERE textileID = NEW.textileID;
    END IF;
END
$$
DELIMITER;
DELIMITER $$
CREATE TRIGGER `update_material_price` BEFORE UPDATE ON `material`
FOR EACH ROW BEGIN
    DECLARE tex_price DECIMAL(10, 2);
    DECLARE acc_price DECIMAL(10, 2);
    SELECT tex_unit_price INTO tex_price
    FROM textile
    WHERE textileID = NEW.textileID;
    SELECT acc_unit_price INTO acc_price
    FROM accessories
    WHERE accessoriesID = NEW.accessoriesID;

```

```
        SET NEW.material_price = (tex_price * NEW.tex_q) + (acc_price  
* NEW.acc_q);  
END  
$$  
DELIMITER;
```